Machine Learning Python Lab Cheat Sheet

Comprehensive reference for core algorithms, model instantiation, training, and objective optimization definitions.

□ 1. Core Workflow Setup & Evaluation

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, classification_report

# Train-Test Split Boilerplate
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling (Essential for Regularization, KNN, Gradient Descent)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

□ 2. Regression Algorithms

A. Linear Regression

- **Objective:** Minimize Residual Sum of Squares (RSS).

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Metrics
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("MSE:", mean_squared_error(y_test, y_pred))
```

B. Polynomial Regression

- **Objective:** Map non-linear relationships into higher-dimensional linear feature spaces.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline

# Create pipeline containing transformation and model steps
degree = 2
poly_model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
poly_model.fit(X_train, y_train)

y_pred = poly_model.predict(X_test)
```

C. Ridge & Lasso Regression (Regularized Linear Models)

- **Ridge (L2 Regression):** Adds a squared penalty (\(||□eta||_2^2\)) to the loss function to shrink coefficients.
- **Lasso (L1 Regression):** Adds an absolute penalty (\(||□eta||_1\)) forcing uninformative feature weights to zero.

```
from sklearn.linear_model import Ridge, Lasso

# Ridge Regression
ridge = Ridge(alpha=1.0) # alpha = regularization strength
ridge.fit(X_train_scaled, y_train)

# Lasso Regression (Useful for intrinsic feature selection)
lasso = Lasso(alpha=0.1)
lasso.fit(X_train_scaled, y_train)
```

□ 3. Classification & Density Estimation

A. Logistic Regression

- **Maximum Likelihood Estimation (MLE):** Standard logistic regression minimizing Cross-Entropy error without weights penalties.
- **Maximum A Posteriori (MAP):** Logistic regression incorporating weight priors (L2 equivalent to Gaussian prior; L1 equivalent to Laplacian prior).

```
from sklearn.linear_model import LogisticRegression

# Logistic Regression via MLE (No penalty / unregularized)
log_mle = LogisticRegression(penalty=None)
log_mle.fit(X_train_scaled, y_train)

# Logistic Regression via MAP (L2 / Ridge regularization penalty)
log_map_l2 = LogisticRegression(penalty='l2', C=1.0) # C = 1/alpha (Inverse regularization strength)
log_map_l2.fit(X_train_scaled, y_train)

# Logistic Regression via MAP (L1 / Lasso regularization penalty)
log_map_l1 = LogisticRegression(penalty='l1', solver='liblinear', C=1.0)
log_map_l1.fit(X_train_scaled, y_train)
```

B. Multinomial Distribution (MLE & MAP)

- Directly parameterizing categorical counts over discrete sample spaces.
- **MLE:** Relative empirical frequencies.
- **MAP:** Incorporates Laplace/Dirichlet smoothing (pseudo-counts) to bypass zero probabilities.

```
# Assuming counts matrix X of shape (n_samples, n_features) and vector target y
# Handled optimally via Naive Bayes estimators targeting categorical spaces

from sklearn.naive_bayes import MultinomialNB

# Multinomial MLE (No smoothing; alpha set to absolute 0)
mn_mle = MultinomialNB(alpha=1e-10) # 0 is unstable, use small epsilon
mn_mle.fit(X_train, y_train)

# Multinomial MAP (Laplace smoothing enabled via alpha=1.0)
mn_map = MultinomialNB(alpha=1.0)
mn_map.fit(X_train, y_train)
```

C. Naive Bayes Classification

```
from sklearn.naive_bayes import GaussianNB

# Gaussian Naive Bayes (Continuous features assuming normality)
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)
```

D. K-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier

# Instantiate model with custom distance metrics
knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2) # p=2 is Euclidean distance
knn.fit(X_train_scaled, y_train)

y_pred = knn.predict(X_test_scaled)
```